

For the above declaration to work, we need to set up the network named *glarimy*. The following does that job.

```
networks:
  glarimy:
    driver: bridge
```

With this, any container on the same *glarimy* network can reach the Zookeeper service using the DNS name *zookeeper*. Consequently, if we run the Kafka on the same network, it can connect to the Zookeeper without any issue.

Just like we declared the Zookeeper, we can also declare the Kafka:

```
kafka:
  image: docker.io/bitnami/kafka:3.1
  ports:
    - "9092:9092"
  volumes:
    - "kafka_data:/glarimy"
  environment:
    - KAFKA_CFG_ZOOKEEPER_CONNECT=zookeeper:2181
    - ALLOW_PLAINTEXT_LISTENER=yes
  networks:
    - glarimy
  depends_on:
    - zookeeper
```

The only additional field in the above declaration that is of interest is the *depends-on*. It tells the *docker-compose* tool to start the *kafka* container only after the *zookeeper* container is started.

Why should we limit ourselves only to the Zookeeper and Kafka? We can also deploy the *AddService* along with them in the same *docker-compose* since they have intimacy.

```
ums:
  image: glarimy/ums-add-service
  networks:
    - glarimy
  depends_on:
    - zookeeper
```

The above is self-explanatory. Following is the full manifest:

```
version: "2"
networks:
  glarimy:
    driver: bridge
services:
  zookeeper:
```

```
  image: docker.io/bitnami/zookeeper:3.8
  ports:
    - "2181:2181"
  volumes:
    - "zookeeper_data:/glarimy"
  environment:
    - ALLOW_ANONYMOUS_LOGIN=yes
  networks:
    - glarimy
kafka:
  image: docker.io/bitnami/kafka:3.1
  ports:
    - "9092:9092"
  volumes:
    - "kafka_data:/glarimy"
  environment:
    - KAFKA_CFG_ZOOKEEPER_CONNECT=zookeeper:2181
    - ALLOW_PLAINTEXT_LISTENER=yes
  networks:
    - glarimy
  depends_on:
    - zookeeper
volumes:
  zookeeper_data:
    driver: local
  kafka_data:
    driver: local
```

Once the Docker images are available, either locally or on the Docker Hub, the following command from the location of *docker-compose.yml* simply deploys them on the Docker environment:

```
sudo docker-compose up -d
```

Use the following command to see if all the three containers are up and running:

```
sudo docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Ports}}\t{{.Status}}"
```

It gives an output similar to Figure 1.

Now, go to the container where the *AddService* is running:

```
sudo docker exec -it f4aa72bcec4a bash
```

Once the *bash* terminal of the container is available, run a *CURL* command to post a new user to the *AddService*.

```
curl -H "Content-Type: application/json" -d '{"name":"Krishna Mohan", "phone":"9731423166"}' http://localhost:8080/user
```